
```
function solve(board, maxMoves) {
    // Attempt to solve the puzzle in `board` in at most `maxMoves`
    // moves. Returns true if solved, otherwise false.
    document.write("Attempting to solve in at most " + maxMoves + " moves...<br /
>");
    let solutionMoves = []; // A list of UP, DOWN, LEFT, RIGHT values.
    let solved = attemptMove(board, solutionMoves, maxMoves, null);
}
```

The `solve()` function has two parameters: `board` contains the data structure of the puzzle to solve, and `maxMoves` is the maximum number of moves the function should make to try to solve the puzzle. The `solutionMoves` list or array contains the sequence of UP, DOWN, LEFT, and RIGHT values that produce the solved state. The `attemptMove()` function modifies this list or array in place as it makes recursive calls. If the initial `attemptMove()` function finds a solution and returns `True`, `solutionMoves` contains the sequence of moves for the solution.

The `solve()` function then makes the initial call to `attemptMove()`, and stores the `True` or `False` it returns in the `solved` variable. The rest of the `solve()` function handles these two cases:

Python

```
if solved:
    displayBoard(board)
    for move in solutionMoves:
        print('Move', move)
        makeMove(board, move)
        print() # Print a newline.
        displayBoard(board)
        print() # Print a newline.

    print('Solved in', len(solutionMoves), 'moves:')
    print(' ', '.join(solutionMoves))
    return True # Puzzle was solved.
else:
    return False # Unable to solve in maxMoves moves.
```

The JavaScript code is as follows:

JavaScript